

实验一

年级：_____2022_____专业：_____计算机科学与技术_____

姓名：_____姚知言_____学号：_____2211290_____

1 实验内容

自行输入两个无序链表 A 和 B，要求均带有头结点。

(1) 将链表 A 升序存储，链表 B 降序存储，并分别输出；然后将 A 和 B 合并为一个无重复元素升序的链表 C，输出链表 C；

(2) 删除链表 C 倒数第 k 个结点，k 要求手动输入；

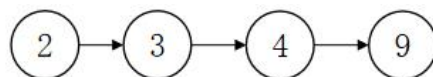
(3) 若链表 D 和 E 满足每个节点存储一位数字，设计算法实现每位数字相加，具体规则如下：

a. 每位按照逆序的方式存储，例如右侧链表表示：9432

b. 将链表 D、E 头节点对齐

c. 按十进制进位相加

d. 以相同形式返回一个表示和的链表



要求：(1) 实验顺序不能颠倒； (2) 每一小问完成后输出结果； (3) 分析时空复杂度。

2 设计思想

该程序的设计思想如下：

根据实验要求，选择单链表思路解决。

首先创建 chain 类和 chainnode 类，分别存储链表和链表的结点。并将 chain 类添加为 chainnode 类的友元，以便通过 chain 类的函数对结点进行操作。

在 chainnode 类中添加私有成员 data 和指针 link，分别存储该结点的数据以及指向下一结点的指针。在 chain 类中添加私有成员 first，存储指向头结点的指针，并重写构造函数，析构函数，以便在定义新 chain 类时生成头结点，在释放 chain 类时依次释放每一个结点。

（注：以下表述中，头结点被认为是第 0 个结点，结点数量不包括头结点）

在 chain 类中定义 insert 函数，实现在第 k 个结点后面添加数据为 x 的结点；定义 output 函数，实现输出整个链表；定义 erase 函数，实现删除倒数第 k 个结点，并返回原来结点的值。

为解决实验内容（1），分别定义 find 函数和 rfind 函数，分别查找链表中存储的第一个比输入值 x 大的结点和输入值比 x 小的结点，同时若将 x 设为该类型所能表达的最大值，find 函数可以等效于输出链表的结点数量。

定义 int 类型 chain 类对象 A、B，通过 find/rfind 函数和 insert 函数，可以完成（1）A、B 链表的构建，并使用 output 函数输出。

在 chain 类中定义 paste 函数，主要功能是若本链表为升序存储，将链表 dest 中的结点依次加入本链表中，并保持升序存储；定义 samebreaker 函数，实现将一个升序/降序存储的链表中存储相同数据的结点唯一化。

定义 int 类型 chain 类对象 C，通过 paste 函数和 samebreaker 函数可以完成（1）C 链表的构建，并使用 output 函数输出。

通过 find 函数（查找链表结点数量）和 erase 函数可以解决（2）的要求。

对于（3），定义 char 类型数组 d、e 承接输入的两串数字，并定义 int 类型 chain 类对象 D、E，使用循环和 insert 函数将数字按照逆序的方式存储进链表。

在 chain 类中定义 plus 函数，将两个链表中数字按照十进制位相加，并存储到新链表中。定义 int 类型 chain 类对象 F 作为和链表，调用 plus 函数可完成 F 链表的构建，并使用 output 函数输出。

3 程序效果

程序的运行效果，输入及输出的相关要求和具体执行结果如下所示：

3.1 输入

(1) in: 9 2 3 4 3 7 2 5 1

(2) in: 3

(3) in: 9432 12357

3.2 输出

(1) out: A:2->3->4->9

B:7->5->3->2->1

C:1->2->3->4->5->7->9

(2) out: 5

(3) out: 9->8->7->1->2

4 核心代码

```
#include<iostream>
#include<cstring>
using namespace std;
template<class T>
class chainnode;
template<class T>
class chain{
public:
    chain();
    ~chain();
    void insert(int k,T x);//将 x 插入到链表的第 k 个结点之后
    void output();//输出链表
    T find(T x);//寻找链表第一个存储比 x 大的数的结点（x 取数值上限时等效查找链表结点数量）
    T rfind(T x);//寻找链表第一个存储比 x 小的数的结点
    void paste(chain<T> &dest);//（需保证本链表原结点按照升序存储）将 dest 链表中的结点依次加入本链表中，并保持升序存储
    T erase(int k);//删除链表中第 k 个结点，并返回原来结点的值
```

void samebreaker(); // (需保证本链表原结点按照升/降序存储) 将存储数据相同的结点唯一化

void plus(chain<T> &p1, chain<T> &p2); // 将 p1, p2 链表中的数据按十进制进位相加

private:

 chainnode<T>* first;
};

template<class T>

class chainnode{
 friend chain<T>;

private:

 T data;
 chainnode<T>* link;
};

template<class T>

chain<T>::chain() {
 first=new chainnode<T>;
 first->link=nullptr;
}

template<class T>

chain<T>::~~chain() {
 chainnode<T>* next;
 while(first) {
 next=first->link;
 delete[] first;
 first=next;
 }
}

template<class T>

```

void chain<T>::insert(int k,T x){
    auto *p=new chainnode<T>;
    p->data=x;
    chainnode<T> *q=first;
    for(int num=0;num<k;num++)
        q=q->link;
    p->link=q->link;
    q->link=p;
}

template<class T>
void chain<T>::output() {
    chainnode<T>* next=first->link;
    while(next) {
        cout<<next->data;
        next=next->link;
        if(next)
            cout<<"->";
    }
    cout<<' \n' ;
}

template<class T>
T chain<T>::find(T x){
    chainnode<T>* next=first->link;
    int n=0;
    while(next) {
        if(next->data<=x)
            n++;
        else
            return n;
    }
}

```

```

        next=next->link;
    }
    return n;
}

template<class T>
T chain<T>::rfind(T x) {
    chainnode<T>* next=first->link;
    int n=0;
    while(next) {
        if(next->data>=x)
            n++;
        else
            return n;
        next=next->link;
    }
    return n;
}

template<class T>
void chain<T>::paste(chain<T>& dest) {
    chainnode<T>* next=dest.first->link;
    while(next) {
        T k=next->data;
        this->insert(this->find(k),k);
        next=next->link;
    }
}

template<class T>
T chain<T>::erase(int k) {
    chainnode<T>* next=first;

```

```

        for(int a=1;a<k;a++)
            next=next->link;
        chainnode<T>* temp=next->link->link;
        T n=next->link->data;
        delete[] next->link;
        next->link=temp;
        return n;
    }

template<class T>
void chain<T>::samebreaker() {
    chainnode<T>* next=first->link;
    int k=1;
    while(next && next->link) {
        if(next->data==next->link->data) {
            erase(k+1);
            continue;
        }
        else {
            k++;
            next = next->link;
        }
    }
}

template<class T>
void chain<T>::plus(chain<T>& p1,chain<T>& p2) {
    chainnode<T> *op1=p1.first,*op2=p2.first;
    int temp=0,n=0;
    do{
        if(op1->link) {

```

```

        op1=op1->link;
        temp+=op1->data;
    }
    if(op2->link){
        op2=op2->link;
        temp+=op2->data;
    }
    insert(n, temp%10);
    n++;
    temp/=10;
}
while(temp||op1->link||op2->link);
}

void op1() {(1)-(2)
    //(1)
    chain<int> A,B;
    for(int num=0;num<4;num++){
        int x;
        cin>>x;
        A.insert(A.find(x), x);
    }
    cout<<"A:";
    A.output();
    for(int num=0;num<5;num++){
        int x;
        cin>>x;
        B.insert(B.rfind(x), x);
    }
    cout<<"B:";

```



```

    B.output();
    chain<int> C;
    C.paste(A);
    C.paste(B);
    C.samebreaker();
    cout<<"C:";
    C.output();

    //(2)

    int m;

    cin>>m;

    cout<<C.erase(C.find(2147483647)-m+1)<<' \n' ;
}

void op2() { //(3)

    chain<int> D,E;
    char d[10],e[10];
    cin>>d>>e;

    for(int num=0;num<strlen(d);num++)
        D.insert(0,d[num]-'0');
    for(int num=0;num<strlen(e);num++)
        E.insert(0,e[num]-'0');

    chain<int> F;

    F.plus(D,E);

    F.output();

}

int main() {

    op1();

    op2();

    return 0;

}

```

时空复杂度分析:

- (1) insert 函数: 空间复杂度 $S(k) = 0$, 时间复杂度 $\theta(k)$
- (2) output 函数: 设结点数量为 n , 空间复杂度 $S(n) = 0$, 时间复杂度 $\theta(n)$
- (3) find, rfind 函数: 设结点数量为 n , 空间复杂度 $S(n) = 0$, 时间复杂度 $O(n)$
- (4) paste 函数: 设目标链表结点数量为 n , dest 链表结点数量为 m , 空间复杂度 $S(n) = n * \text{sizeof}(\text{chainnode})$, 时间复杂度 $O(m(m+n))$
- (5) erase 函数: 设结点数量为 n , 空间复杂度 $S(n) = 0$, 时间复杂度 $O(n)$
- (6) samebreaker 函数: 设结点数量为 n , 空间复杂度 $S(n) = 0$, 时间复杂度 $O(n^2)$
- (7) plus 函数: 设 $p1, p2$ 结点数量 $n1, n2$, $\max(n1, n2) = n$, 空间复杂度 $S(n) = 0$, 时间复杂度 $O(n)$
- (8) 设 A, B 结点数量 $n1, n2$, op1 整体空间复杂度 $S = 2 * (n1 + n2) * \text{sizeof}(\text{chainnode})$, 时间复杂度 $O(n^2)$
- (9) 设 D, E 长度分别为 $n1, n2$, op2, $\max(n1, n2)$ 整体空间复杂度 $S = (n1 + n2 + n) * \text{sizeof}(\text{chainnode})$, 时间复杂度 $O(n)$

5 总结

在本次实验中, 主要复习了 c++ 的基本语法, 熟悉了链表的操作和时空复杂度的计算。了解到线性表在解决各类问题中的应用实例, 明晰线性表在解决各类问题中的重要意义。

